PROJECT 1

Smart Environmental Monitor

Microcontroller Basics

Connecting a simulated microcontroller to digital and analog sensors to sample, process, and display ambient environmental data in real time — built and circuit-tested entirely in Tinkercad.

Name	**Noor Fatima**
Student / Intern ID	**IoT Intern**
Company	**Decode Labs**
Date	**20th June 2026**

Table of Contents

1. Project Overview & Goal

> *Goal: Connect a physical or simulated microcontroller to a digital sensor to read and process ambient environmental data.*

This project introduces the fundamentals of microcontroller-based sensing: wiring a sensor to a development board, sampling data on a fixed interval, and rendering that data somewhere a human can read it. It is the foundation for almost every IoT device — from smart thermostats to industrial monitoring systems — and mirrors the device-layer concepts covered in our broader IoT architecture work.

The intended hardware for this exercise is a DHT11 or DHT22 digital temperature and humidity sensor wired to an ESP32 or Arduino board. Because this project was built and tested inside Tinkercad Circuits rather than on physical hardware, the implementation includes one important adaptation, detailed in Section 3.

2. Key Requirements

Hardware Wiring

Wire a DHT11/DHT22-equivalent sensor setup to an ESP32 or Arduino development board.

Execution Loop

Implement `void loop()` to sample temperature and humidity data every 2 seconds.

Data Output

Render the raw data stream clearly on a local I2C LCD screen or the Serial Monitor.

Key Skills Targeted

- Microcontroller GPIO Pinning
- I2C Protocol
- Digital Sensor Protocols
- Serial Data Debugging
- Analog-to-Digital Conversion

3. The Tinkercad Library Limitation & Our Workaround

> **Important note:** As of this build, Tinkercad Circuits' component library does **not** include a DHT11 or DHT22 sensor part. To complete this project entirely inside the Tinkercad simulator, we engineered a substitute circuit that reproduces the same two data streams — temperature and humidity — using parts that *are* available in the library.

Our Substitution Strategy

Rather than skipping the simulation step, we built an analog stand-in using two components:

- **A potentiometer**, whose knob position produces a variable analog voltage. We treat this voltage as a stand-in for **temperature** — turning the knob simulates the temperature rising or falling, the same way a real DHT sensor's reading would drift.
- **A TMP36 analog temperature sensor**, repurposed as a stand-in for **humidity**. Its output voltage is read on a second analog pin and re-mapped in firmware to a 0–100% humidity range instead of a temperature range.

Both components are read using `analogRead()` on the microcontroller's ADC pins, then converted with `map()` into the same units and variable names a real DHT11/DHT22 would produce. This means the rest of the sketch — the sampling loop, the Serial/LCD output formatting — is written *exactly* as it would be for genuine hardware. Only the raw-value acquisition step differs.

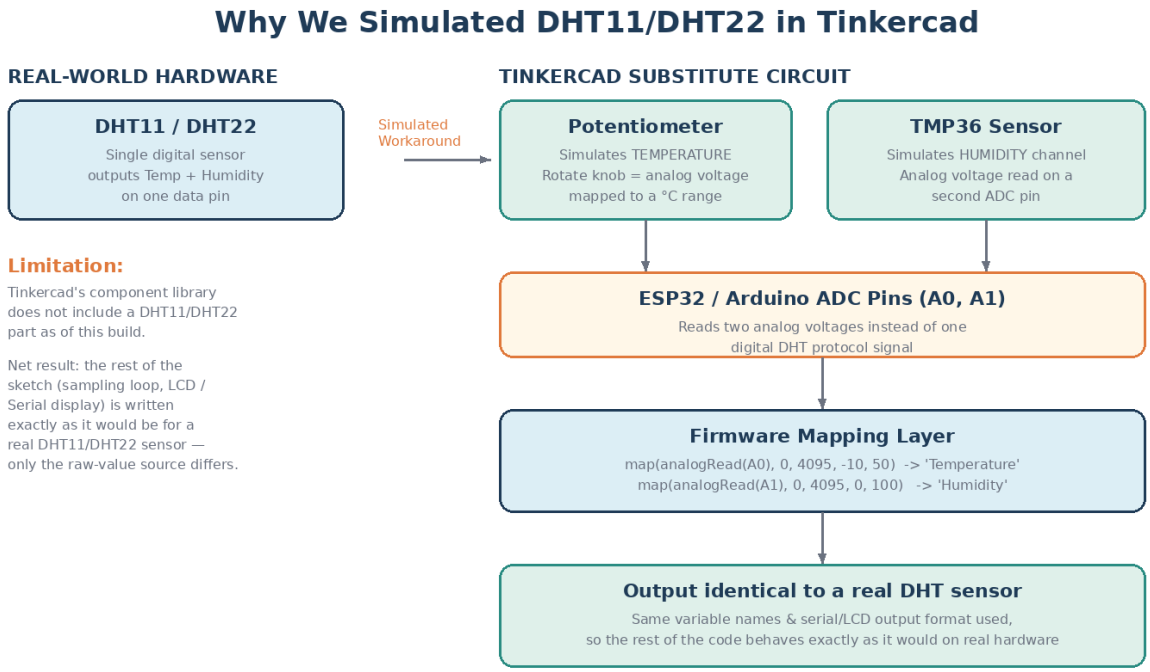


Figure 3.1 — How the potentiometer + TMP36 substitute reproduces DHT11/DHT22-style output

Real DHT11/DHT22 Behavior	Tinkercad Substitute Behavior
Single digital pin, proprietary timing protocol	Two analog pins (A0, A1), standard ADC read
Returns temperature (°C) directly	Potentiometer voltage mapped to a -10°C to 50°C range
Returns humidity (%) directly	TMP36 voltage mapped to a 0–100% range
Reading drifts based on real environment	Reading changes manually (knob) or via TMP36's ambient response

4. Circuit Design & Pin Mapping

The diagram below summarizes how the substitute sensors and the I2C LCD are wired into the ESP32 development board inside Tinkercad. Exact GPIO numbers should be confirmed against your specific board layout, since pin labelling can vary slightly between simulated board models.

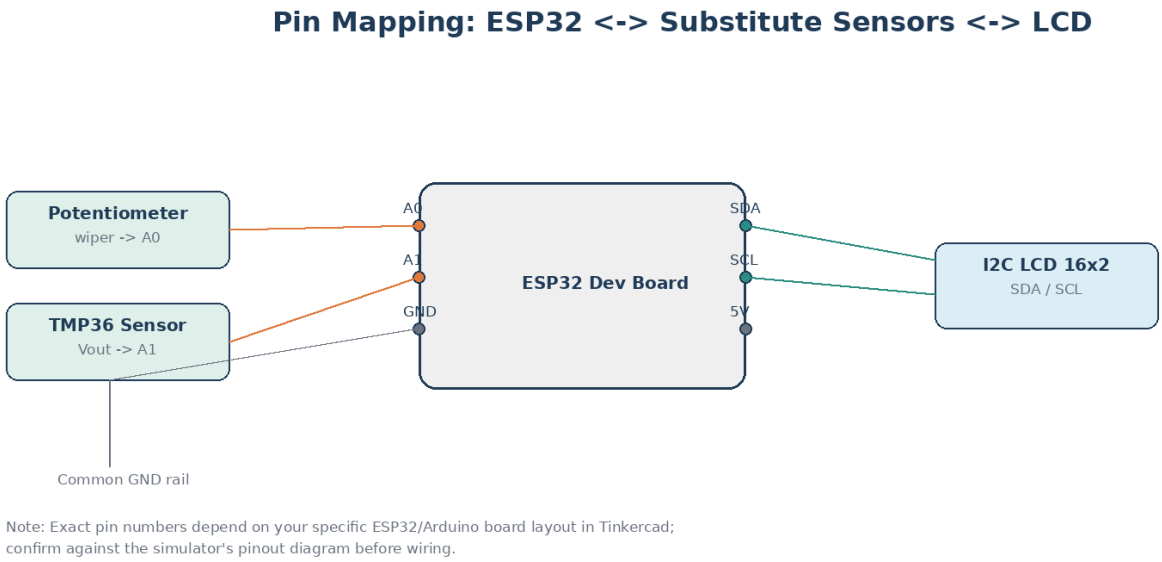


Figure 4.1 — Pin mapping between sensors, ESP32, and the I2C LCD

Connection Summary

Component	Pin	Connects To	Purpose
Potentiometer	Wiper (middle pin)	ESP32 A0	Simulated temperature input
TMP36 Sensor	Vout	ESP32 A1	Simulated humidity input
I2C LCD 16x2	SDA	ESP32 SDA	Display data line
I2C LCD 16x2	SCL	ESP32 SCL	Display clock line
All components	GND	Common ground rail	Shared reference voltage

5. Firmware: Sampling Loop & Display Logic

The firmware below performs three things every 2-second cycle: reads both analog substitute-sensor pins, maps the raw ADC values into temperature/humidity units, and renders the result to both the Serial Monitor and an I2C 16x2 LCD.

5.1 Library Setup & Pin Definitions

```
// Project 1: Smart Environmental Monitor (Tinkercad Simulation)
// Substitute sensors: Potentiometer = Temperature, TMP36 = Humidity

#include <Wire.h>
#include <LiquidCrystal_I2C.h>

LiquidCrystal_I2C lcd(0x27, 16, 2);  // I2C address, columns, rows

const int TEMP_PIN = A0;   // Potentiometer -> simulated temperature
const int HUMIDITY_PIN = A1; // TMP36 -> simulated humidity

const unsigned long SAMPLE_INTERVAL = 2000; // 2 seconds
unsigned long lastSampleTime = 0;
```

5.2 Setup Function

```
void setup() {
  Serial.begin(9600);
  lcd.init();
  lcd.backlight();
  lcd.setCursor(0, 0);
  lcd.print("Env Monitor v1");
  delay(1500);
  lcd.clear();
}
```

5.3 Execution Loop

```
void loop() {
  unsigned long currentTime = millis();

  if (currentTime - lastSampleTime >= SAMPLE_INTERVAL) {
```

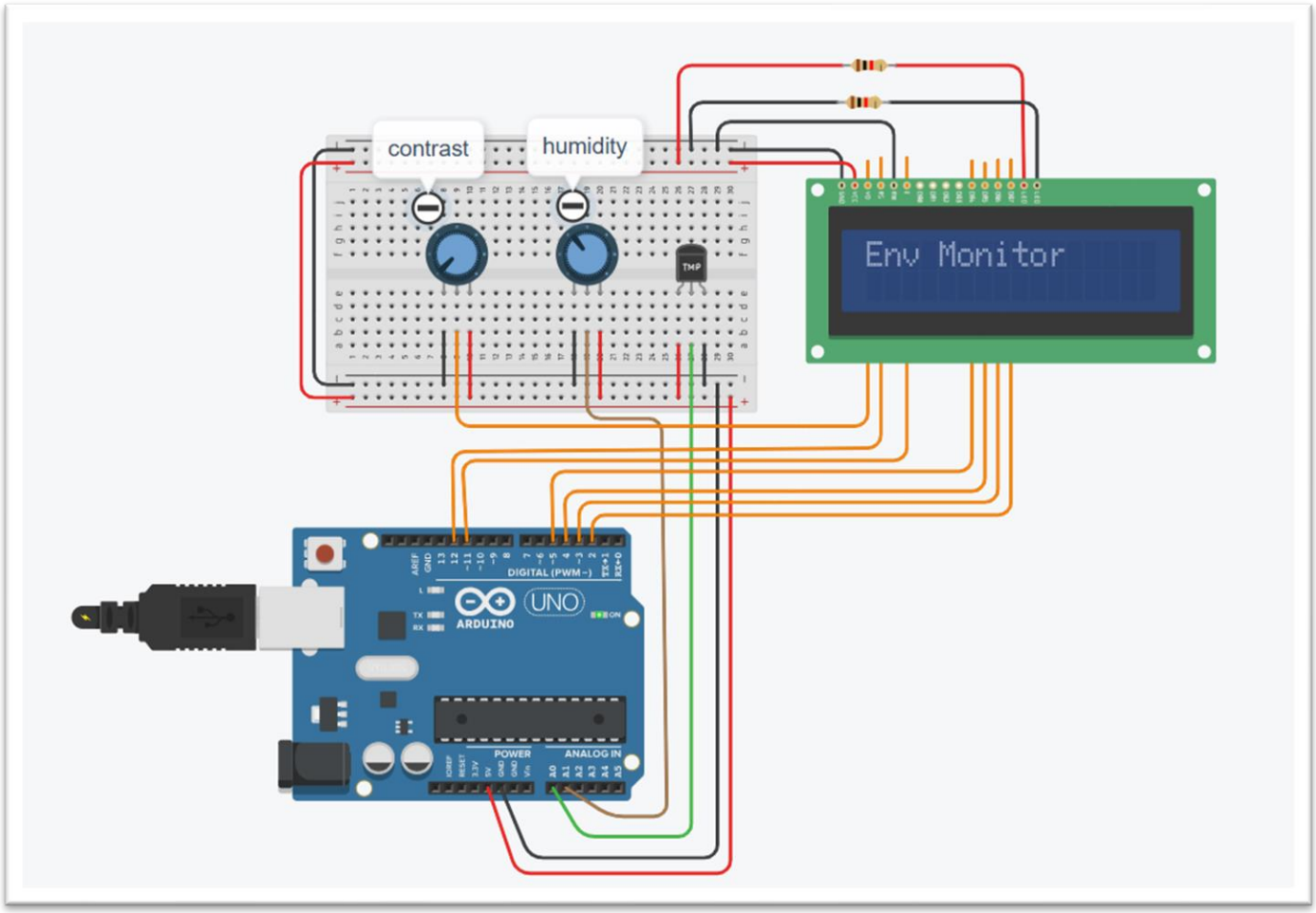
```
    lastSampleTime = currentTime;

    // --- Read raw analog values from substitute sensors ---
    int rawTemp = analogRead(TEMP_PIN);
    int rawHumidity = analogRead(HUMIDITY_PIN);

    // --- Map raw ADC values (0-4095) into realistic ranges ---
    float temperature = map(rawTemp, 0, 4095, -10, 50);
    float humidity = map(rawHumidity, 0, 4095, 0, 100);

    // --- Output to Serial Monitor for debugging ---
    Serial.print("Temperature: ");
    Serial.print(temperature);
    Serial.print(" C   Humidity: ");
    Serial.print(humidity);
    Serial.println(" %");

    // --- Output to I2C LCD screen ---
    lcd.setCursor(0, 0);
    lcd.print("Temp: ");
    lcd.print(temperature);
    lcd.print(" C   ");

    lcd.setCursor(0, 1);
    lcd.print("Hum:  ");
    lcd.print(humidity);
    lcd.print(" %  ");
  }
}
```

Debugging tip: Keep the Serial Monitor open at `9600` baud while testing in Tinkercad. If LCD characters appear garbled, double-check the I2C address (commonly `0x27` or `0x3F`) using an I2C scanner sketch before troubleshooting the wiring.

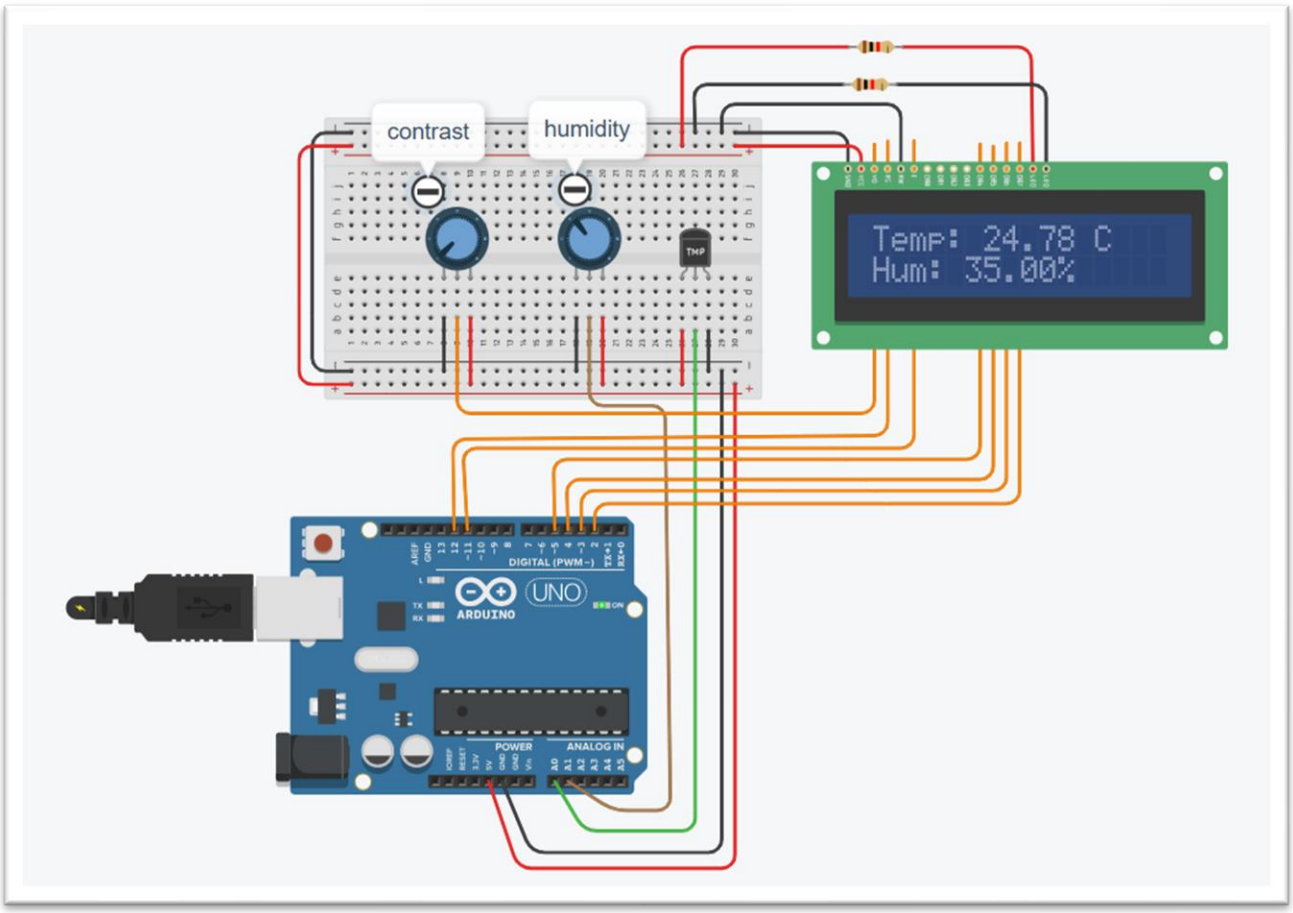
6. Tinkercad Circuit Screenshots

The following spaces are reserved for your own Tinkercad circuit screenshots. Capture the full circuit view, a close-up of the sensor wiring, and a close-up of the LCD/Serial output while the simulation is running.

6.1 Full Circuit View



6.2 LCD / Serial Monitor Output



7. Testing & Observations

7.1 Test Procedure

1. Start the Tinkercad simulation and confirm the LCD initializes with the startup message.
2. Rotate the potentiometer knob and verify the displayed "temperature" value changes accordingly.
3. Confirm the "humidity" value updates independently, sourced from the TMP36 sensor.
4. Verify that both values refresh on a steady 2-second interval — not faster, not slower.
5. Cross-check the LCD output against the Serial Monitor output for consistency.

8. Key Skills Demonstrated

Skill	Where It Was Applied
Microcontroller GPIO Pinning	Mapping analog sensor outputs to specific ESP32 ADC pins (A0, A1)
I2C / Digital Sensor Protocols	Wiring and addressing the I2C LCD; understanding why DHT sensors use a single-wire digital protocol
Serial Data Debugging	Using the Serial Monitor to validate sensor readings independently of the LCD
Analog-to-Digital Conversion	Using `analogRead()` and `map()` to convert raw ADC values into meaningful units
Timed Execution Loops	Using `millis()` for non-blocking 2-second sampling instead of `delay()`

9. Conclusion

This project demonstrated the core mechanics of microcontroller-based environmental sensing — wiring a sensor to a development board, sampling on a fixed interval, and rendering live data to a human-readable output. The unavailability of a DHT11/DHT22 part in Tinkercad's library required an engineering workaround rather than abandoning the simulation: combining a potentiometer and a TMP36 sensor to recreate the same dual-channel data stream a real DHT sensor would provide, without changing how the rest of the firmware is written.

This same pin-mapping, sampling-loop, and I2C display pattern carries forward directly into later projects that use real sensor hardware, the only change being the data acquisition method itself.

— End of Report —